Voicu Alexandru Iulian
333AA

Documentație Proiect: Sistem de Gestiune Coworking

Tehnologii: Spring Boot (Java), React.js, SQL Nativ (PostgreSQL/MySQL).

1. Descrierea Cerințelor

Aplicația are scopul de a digitaliza procesul de administrare al unui spațiu de coworking. Cerințele principale includ:

- **Gestiunea Utilizatorilor:** Adăugarea, vizualizarea, căutarea și ștergerea clienților folosind interogări SQL native.
- **Administrarea Spațiilor:** Monitorizarea resurselor disponibile (birouri, săli de conferințe) și a prețurilor acestora.
- **Sistem de Rezervări:** Corelarea utilizatorilor cu spațiile disponibile pe perioade determinate de timp.
- **Procesarea Plăților:** Înregistrarea tranzacțiilor și monitorizarea venitului total (Profit).
- **Rapoarte Administrative:** Implementarea unor funcționalități complexe pentru identificarea clienților premium și a spațiilor cu cerere ridicată prin subinterogări SQL

2. Etapa de Proiectare

Aplicația respectă arhitectura de tip **Client-Server**:

A. Arhitectura Backend (Spring Boot)

Proiectul este structurat pe patru straturi pentru o mentenanță ușoară:

1. **Model (Entity):** Definește structura obiectelor (Utilizator, Rezervare, Plata).
2. **Repository:** Conține interogările SQL Native folosind adnotarea @Query.
3. **Service:** Gestionează logica de business (ex: calculul profitului, procesarea autentificării).
4. **Controller:** Expune rutele API (REST) către frontend (ex: /api/utilizatori/cauta).

B. Arhitectura Frontend (React)

Interfața este modulară, construită din componente independente:

- UtilizatoriTable.js, RezervariTable.js, PlatiTable.js.
- Utilizarea **Hooks** (useState, useEffect) pentru gestionarea stării și a datelor primite de la server.

3. Descrierea Tabelelor și a Relațiilor

Tabele Principale:

- **Utilizatori:** Stochează datele de identificare (Nume, Prenume, Email, Parolă) și rolul (ADMIN/CLIENT).
- **Spații:** Conține detaliile despre locurile de închiriat (Capacitate, Preț pe oră) și tipul acestora.
- **Rezervări:** Tabelul central care face legătura între utilizatori și spații pentru un anumit interval de timp.
- **Plăți:** Înregistrează detaliile financiare (Suma, Metoda de plată, Data plății) asociate fiecărei rezervări.

Relațiile dintre tabele:

- **Utilizatori – Rezervări (1:N):** Un utilizator poate efectua mai multe rezervări, dar o rezervare aparține unui singur utilizator.

- **Spații – Rezervări (1:N):** Un spațiu poate fi subiectul mai multor rezervări în perioade diferite.
- **Rezervări – Plăți (1:1):** Fiecare rezervare are asociată o singură plată unică.
- **Tipuri Spații – Spații (1:N):** Un tip (birou, sală) poate fi atribuit mai multor spații.
- **Spații – Facilități (N:N):** Un spațiu poate avea multiple facilități, iar o facilitate poate fi prezentă în mai multe spații (gestionat prin tabelul de legătură Spații facilități).

4. Constrângeri de Integritate Impuse

Pentru a garanta validitatea și securitatea datelor, au fost implementate următoarele constrângeri:

- **Primary Key (PK):** Fiecare tabel are un identificator unic (ex: ID Utilizator, ID Rezervare) pentru a preveni înregistrările duplicate.
- **Foreign Key (FK):** Asigură coerența legăturilor (ex: nu se poate crea o rezervare pentru un utilizator care nu există).
- **UNIQUE:** Coloana Email din tabelul Utilizatori și ID Rezervare din Plăți sunt unice.
- **CHECK:** Validează datele numerice (ex: Capacitate >= 0, Preț pe oră >= 0, Suma >= 0) și limitează valorile pentru Rol sau Status.
- **NOT NULL:** Câmpurile esențiale (Nume, Email, Parolă, Data start) trebuie completate obligatoriu.

5. Funcționarea Aplicației

Funcționarea sistemului de gestiune Coworking se bazează pe o interacțiune fluidă între interfața utilizatorului (Frontend) și serverul de aplicație (Backend), asigurând procesarea datelor în timp real prin interogări SQL native.

5.1. Fluxul de Gestiune a Utilizatorilor

- **Afișarea Datelor:** La accesarea modulului "Utilizatori", aplicația execută automat o cerere de tip GET. Serverul procesează această cerere apelând metoda findAllNative() din Repository, care returnează lista completă a utilizatorilor direct din baza de date.
- **Adăugarea Nativă:** Administratorul poate introduce un utilizator nou completând câmpurile Nume, Prenume și Email. La apăsarea butonului "Salvează Nativ", sistemul execută o comandă SQL de tip INSERT, generând automat o parolă implicită și atribuind rolul de "CLIENT" conform constrângerilor de tip DEFAULT.
- **Căutarea Dinamică:** Funcția de căutare permite filtrarea utilizatorilor după nume. React trimite valoarea introdusă către ruta /api/utilizatori/cauta/{nume}, unde backend-ul rulează o interogare cu operatorul LIKE, permițând identificarea rapidă a înregistrărilor fără a reîncărca întreaga pagină.

5.2. Gestiunea Rezervărilor și a Spațiilor

- **Corelarea Datelor:** Utilizatorul selectează un spațiu disponibil (ex: Birou, Sală de conferințe). Aplicația verifică integritatea referențială prin cheile externe (ID Utilizator, ID Spațiu), asigurându-se că rezervarea este legată corect de un client existent și de o resursă validă.
- **Sistemul de Statusuri:** Fiecare rezervare este monitorizată printr-un sistem de statusuri colorate condiționat în interfață (ex: **CONFIRMATA** - verde, **IN ASTEPTARE** - galben, **ANULATA** - roșu). Această vizualizare oferă administratorului o imagine clară asupra ocupării spațiilor în timp real.
- **Căutarea după Email:** În tabelul de rezervări, administratorul poate filtra rezultatele introducând adresa de email a clientului. Această funcționalitate utilizează o interogare SQL complexă de tip JOIN între tabelele Rezervari și Utilizatori pentru a returna doar înregistrările asociate acelui email.

5.3. Monitorizarea Financiară și Profitul

- **Procesarea Plăților:** Pentru fiecare rezervare confirmată, sistemul generează o înregistrare în tabelul Plati, respectând relația de tip 1:1. Suma

este validată prin constrângerea de tip CHECK >= 0 pentru a preveni erorile de introducere a datelor.

- **Banner-ul de Profit:** În partea superioară a modulului de plăți, aplicația afișează "Venit Total Generat". Această valoare este calculată printr-o funcție de agregare SQL (SUM) executată la nivel de baza de date, oferind o statistică financiară actualizată instantaneu.

5.4. Execuția Rapoartelor Complexe (SQL Subqueries)

Aplicația include un modul de raportare avansată care utilizează subinterogări SQL pentru a extrage informații de nișă:

- **Identificarea Datelor Premium:** Sistemul compară prețul fiecărui spațiu cu media aritmetică a tuturor spațiilor. Dacă prețul este superior mediei, rezervarea este marcată ca fiind "Premium".
- **Filtrarea Clienților VIP:** Prin utilizarea unei subinterogări corelate, aplicația identifică utilizatorii care au cumulat plăți peste un anumit prag valoric, facilitând programele de fidelizare a clienților.

6. Interogări Simple și Complexe (SQL Nativ)

1. Interogări Simple (cu JOIN și Parametri)

Acestea sunt "motorul" tabelelor tale principale. Rezultatul lor se vede imediat ce deschizi paginile respective în React.

1. **Vizualizarea Spațiilor cu Categorie (JOIN)**

```
25      // ========================================
26
27      // Catalogul spatiilor cu denumirea categoriei (JOIN)
28      @Query(nativeQuery = true, value =  1 usage
29              "SELECT s.* FROM Spatii s JOIN Tipuri_spatii ts ON s.id_tip = ts.id_tip")
30      List<Spatiu> findAllNative();
31
```

2. **Vizualizarea Recenziilor cu Detalii Utilizator (JOIN)** –

Voicu Alexandru Iulian
333AA

```java
public interface RecenzieRepository extends JpaRepository<Recenzie, Integer> {

    // 1. VIZUALIZARE NATIVĂ (JOIN) - Afișează recenziile cu numele spațiului și al utilizatorului
    @Query(nativeQuery = true, value =  1 usage
            "SELECT r.* FROM Recenzii r " +
                    "JOIN Spatii s ON r.id_spatiu = s.id_spatiu " +
                    "JOIN Utilizatori u ON r.id_utilizator = u.id_utilizator")
    List<Recenzie> findAllNative();
```

3. **Căutarea Utilizatorilor după Nume (Parametru + Filtrare)** - *Rezultatul butonului de căutare*

```java
    // 8. CĂUTARE DUPĂ NUME (Rezolvă eroarea din React)
    @Query(nativeQuery = true, value = "SELECT * FROM utilizatori WHERE LOWER(nume) LIKE LOWER(CONCAT('%', :nume, '%'))")  1 usage
    List<Utilizator> findByNumeNative(@Param("nume") String nume);
}
```

4. **Clasificarea Spațiilor pe Categorii (JOIN)** - *Vizibilă în panoul de administrare a spațiilor*

```java
    // Catalogul spatiilor cu denumirea categoriei (JOIN)
    @Query(nativeQuery = true, value =  1 usage
            "SELECT s.* FROM Spatii s JOIN Tipuri_spatii ts ON s.id_tip = ts.id_tip")
    List<Spatiu> findAllNative();
```

5. **Calculul Profitului Total (Agregare)** - *Vizibil în banner-ul verde de sus*

```java
    // Calcul profit total pentru banner-ul de sus
    @Query(nativeQuery = true, value = "SELECT COALESCE(SUM(suma), 0) FROM plati")  1 usage
    Double getProfitTotalComplex();
```

6. **Căutarea Rezervărilor după Email (JOIN + Parametru)** - *Rezultatul lupei din Rezervări*

```java
    // Pentru căutarea după email în tabelul de Rezervări
    @Query(nativeQuery = true, value = "SELECT r.* FROM rezervari r JOIN utilizatori u ON r.id_utilizator = u.id_utilizator WHERE u.email LIKE %:email%")  1 usage
    List<Rezervare> findByUtilizatorEmailNative(@Param("email") String email);
```

2. Interogări Complexe (Subqueries)

Acestea sunt folosite pentru secțiunea de "Rapoarte SQL" și analize avansate.

Voicu Alexandru Iulian
333AA

1. **Raport Clienți Premium (Subquery Corelat)**
   - Identifică utilizatorii care au plătit în total peste 500 RON.

```
// 7. Raport Clienti Premium (> 500 RON)
@Query(nativeQuery = true, value =  1 usage
        "SELECT u.* FROM utilizatori u WHERE " +
                "(SELECT SUM(p.suma) FROM plati p JOIN rezervari r ON p.id_rezervare = r.id_rezervare " +
                "WHERE r.id_utilizator = u.id_utilizator) > 500")
List<Utilizator> findPremiumCustomersNative();
```

2. **Raport Spații Premium (Subquery)**
   - Selectează rezervările făcute pentru spații care costă mai mult decât media pieței.

```
16
17      // Pentru Rapoarte Complexe: Rezervări unde pretul spatiului > media
18      @Query(nativeQuery = true, value = "SELECT * FROM rezervari WHERE id_spatiu IN (SELECT id_spatiu FROM spatii WHERE pret_pe_ora > (SELECT AVG(pret_pe_ora) FROM spatii))")
19      List<Rezervare> findPremiumReservationsNative();
20  }
```

3. **Identificare Spații fără Feedback (Subquery cu NOT IN)**
   - Util pentru administratori să vadă ce spații nu au fost încă evaluate.

```
// Identificare spatii fără feedback (Subquery cu NOT IN)
@Query(nativeQuery = true, value =  1 usage
        "SELECT * FROM Spatii WHERE id_spatiu NOT IN (SELECT id_spatiu FROM Recenzii)")
List<Spatiu> findSpatiiFaraRecenzii();
```

4. **Recenzii peste Media de Rating (Subquery cu Agregare)**
   - Filtrează doar cele mai bune recenzii din sistem.

```
    // 3. INTEROGARE COMPLEXĂ (Subquery + Agregare) - Recenziile cu rating peste medie
    @Query(nativeQuery = true, value =  no usages
            "SELECT * FROM Recenzii WHERE rating > (SELECT AVG(rating) FROM Recenzii)")
    List<Recenzie> findBestReviewsNative();
}
```